

COMPLEX ENGINEERING ACTIVITY (CEA)

ECAT Exam Application — Dual Portal System

A Python Console-Based Examination Management System

Student: Abdul Moqet | Roll No: 101 | CYS(S)-2026-101

1. Project Overview

The ECAT Exam Application is a fully functional, console-based examination management system developed in Python. It simulates a real-world dual-portal exam environment — one for students and one for administrators — closely modelled on Pakistan's Engineering Council Admission Test (ECAT) workflow. The application manages the complete life-cycle of a test session: authentication, question presentation, answer collection, auto-grading, result storage, and post-exam analytics.

Attribute	Detail
Language	Python 3.x (standard library only)
Interface	Command-Line Interface (CLI)
Total Lines of Code	330
Questions Bundled	10 (Math, Physics, Chemistry, English, Computer)
Marking Scheme	+4 correct / -1 wrong / 0 skipped
Portals	Student Portal & Admin Portal
Authentication	Username + Password with 3-attempt logout

2. System Architecture & Module Breakdown

The application follows a procedural, function-based architecture. All logic is encapsulated in well-named functions invoked from a central `main()` control loop. The structure is divided into five functional layers:

Layer / Module	Functions / Constants	Responsibility
Configuration	ADMIN_USER, ADMIN_PASS, STUDENT_USER, STUDENT_PASS, CORRECT_MARKS, WRONG_MARKS, SKIP_MARKS, MIN_QUESTIONS	Stores all credentials and marking constants in one place for easy management
Data Store	questions[] list all_results[] list	In-memory dictionaries hold the question bank and all student results
Utility	grade(percent)	Pure function; maps a percentage to a letter grade label (EXCELLENCE, GOOD, etc.)
Student Portal	exam() show_rules() student_login()	Handles student authentication, exam delivery, answer capture, scoring, and result display

Admin Portal	admin_login() admin_menu() view_all_questions() add_question() delete_question() question_bank_stats() view_all_results() view_detailed_result() class_result_stats()	Full CRUD on question bank plus comprehensive result analytics for
Entry Point	main()	Top-level loop presenting the dual-portal menu and routing control to

3. Student Portal — Detailed Walkthrough

3.1 Authentication (student_login)

The student logs in with a fixed username/password pair (*student* / *student123*). A counter limits failed attempts to three; on the third failure the account is locked for the session. On success the student menu offers three options: start the exam, view rules, or log out.

3.2 Exam Engine (exam)

The exam function orchestrates the entire test session. It collects the student's full name and roll number, then iterates over every question in the global list, presenting the subject tag, question text, and four labelled choices (A-D).

- Input validation: only A, B, C, D, S (skip), or SUBMIT are accepted; any other input triggers a re-prompt.
- Early exit: typing SUBMIT at any prompt immediately ends the exam.
- Skipped questions: stored as 'S' and receive zero marks.
- Answer dictionary: keys are 0-based question indices; values are the student's response letter.

3.3 Scoring Algorithm

After submission the engine tallies correct, wrong, and skipped counts, then applies the following formula:

```
score    = (correct x +4) + (wrong x -1) + (skipped x 0)
percent  = (score / total_marks) x 100    [total_marks = questions x 4]
```

The result dictionary — including all per-question answers — is appended to **all_results** so the admin can retrieve it later.

3.4 Result & Answer Review

Immediately after scoring the student sees a formatted result card (name, roll, score, percentage, grade, date/time) followed by a per-question review table showing: question number, subject, the student's answer, the correct answer, and the outcome label (CORRECT / WRONG / SKIPPED).

4. Admin Portal — Detailed Walkthrough

4.1 Authentication (admin_login)

Admins authenticate with `ecat_admin / ecat@2024`. The same 3-attempt lockout policy applies. A successful login routes to the 8-item admin menu.

4.2 Question Bank Management

Feature	Function	Description
View All Questions	<code>view_all_questions()</code>	Prints every question with its choices and correct answer.
Add New Question	<code>add_question()</code>	Interactively collects subject, question text, four choices, and the correct letter.
Delete a Question	<code>delete_question()</code>	Displays the numbered list, prompts for a number, and pops that entry from the list.
Question Bank Statistics	<code>question_bank_stats()</code>	Shows total count and a per-subject breakdown.

4.3 Result Analytics

Three analytics functions give the admin a complete picture of class performance:

- **View All Student Results** — tabular summary: name, roll, score, percentage, grade, date/time.
- **View Detailed Result** — drills into one student's record showing the full per-question breakdown (question text truncated to 33 chars, given answer, correct answer, outcome).
- **Class Result Statistics** — aggregate metrics: highest/lowest/average score, pass/fail count (threshold 50%), and grade distribution across all four bands.

5. Data Structures

5.1 Question Dictionary

Each entry in the `questions` list is a Python dictionary with five keys:

```
{
  "id"      : 1,
  "subject" : "Math",
  "question": "5 * 2 + 3 = ?",
  "choices" : ["A. 16", "B. 11", "C. 13", "D. 5"],
  "answer"  : "C"
}
```

5.2 Result Dictionary

Each entry in the `all_results` list stores everything needed for review:

```
{
  "Name"      : "John Doe",
  "Roll No"   : "2024-EE-01",
  "Score"     : 28,
  "Total"     : 40,
  "Correct"   : 8,
  "Wrong"     : 4,
  "Skipped"   : 2,
  "Percentage": 70.0,
  "Grade"     : "GOOD",
  "Date/Time" : "08-Jun-2025 02:30 PM",
  "Answers"   : {0: "C", 1: "A", 2: "S", ...}
}
```

6. Marking Scheme & Grading Rubric

Event	Marks	Grade Band	Percentage Range	Label
Correct Answer	+4	Band A	80% and above	EXCELLENT
Wrong Answer	-1	Band B	65% - 79%	GOOD
Skipped / No Answer	0	Band C	50% - 64%	AVERAGE
		Band D	Below 50%	BELOW AVERAGE

7. Conclusion

The ECAT Exam Application — Dual Portal System successfully demonstrates core software-engineering principles within a pure Python, zero-dependency context. It cleanly separates concerns between student-facing and admin-facing functionality, implements a negative-marking grading algorithm aligned with real ECAT conventions, and provides actionable analytics for educators. The project serves as a solid foundation that can be extended with persistent storage, enhanced security, and a richer user interface to evolve into a production-grade examination platform.

Submitted by: Abdul Moqheet | Roll No: 101 | CYS(S)-2026-101